

Lezione 13 - Ricorsione

Corso — Programmazione

Marco Anisetti

e-mail: marco.anisetti@unimi.it

web: <http://homes.di.unimi.it/anisetti/>

Programmi basati su relazioni di ricorrenza(1)

- La ricorrenza può avere due forme nella programmazione, le ripetizioni (ciclo) e la ricorsione
- Esempio ripetizioni semplici nella forma pre-condizionata **while** *cond* **do** *I*
- Le ripetizioni hanno senso se terminano ovvero se *I* modifica almeno una variabile che compare in *cond* rendendola non più soddisfatta
- Il ciclo sopra può diventare logicamente il seguente: $V \leftarrow v_0$; **while** $p(V)$ **do** $V \leftarrow f(V)$
- Dove p è un predicato (booleano) e f una funzione
- All'inizio della ripetizione V deve avere un valore ben determinato V_0

Programmi basati su relazioni di ricorrenza(2)

- Esempio: Calcolo fattoriale $f(n) = 1 * 2 * 3 * \dots * n = n!$
($n \geq 0$)
- E' calcolabile usando una relazione di ricorrenza
- $f(n) = n * f(n - 1)$ per $n > 0$
- Avendo come condizione iniziale $f(0) = 1$

Ricorsione

Funzione ricorsiva è una funzione che chiama se stessa.

Come per il ciclo stabilisce una relazione di ricorrenza, andando ad eseguire ripetutamente il corpo (o parti del corpo) della funzione stessa, ovvero ripetendo sempre le stesse operazioni per un certo numero di volte.

Si noti che la funzione e la stessa, le chiamate sono diverse

La ricorsione è l'analogo informatico (ossia computazionale) **della induzione matematica.**

La definizione induttiva di fattoriale

$$n! := \begin{cases} 1 & \text{se } n = 0 \\ n(n-1)! & \text{se } n \geq 1 \end{cases}$$

$N=0$ si chiama **caso base**
(non contiene rimandi
alla funzione fattoriale)
 $N>0$ **caso induttivo o passo**

Esempio fattoriale

Se scriviamo `fatt (n)` per il fattoriale di n , usando l'ordinaria notazione funzionale, la definizione diventa:

$$\text{fatt}(n) := \begin{cases} 1 & \text{se } n = 0 \\ n \text{fatt}(n - 1) & \text{se } n \geq 1 \end{cases}$$

Se poi interpretiamo questa definizione come la specifica di un algoritmo per calcolare il fattoriale, vediamo che il prototipo della funzione che implementa l'algoritmo è

```
int fatt(int n)
```

Il corpo della funzione richiamerebbe se stessa per seguire la definizione induttiva

Esempio fattoriale

Una funzione ricorsiva deve contenere un blocco di selezione con

1. un caso base che calcola esplicitamente un risultato
2. un caso ricorsivo che "riduce" i dati e riapplica la funzione

```
int fattoriale (int n)  
{  
  if (n <= 1) /* selezione */  
    return 1; /* 1. caso base */  
  else  
    return n * fattoriale(n - 1); /* 2. caso ricorsivo */  
}
```

Caso base e riduzione

Il caso base corrisponde a dati per cui è facile trovare il risultato (si scrive direttamente del codice per trovarlo)

Il caso ricorsivo corrisponde a dati per cui è difficile trovare il risultato, ma diventa facile conoscendo il risultato di problemi "più piccoli"

- si costruiscono i dati di tali problemi e si richiama la funzione

Per funzionare, il metodo richiede

1. un algoritmo per il caso base
2. una procedura di "riduzione" dei dati (winding)
3. la garanzia che i dati ricadano prima o poi nel caso base
4. una procedura di "ricostruzione" del risultato (unwinding)

Potenza

Calcolo della potenza n-esima di un numero intero i

- **caso base (n = 0):** la potenza è 1 per qualsiasi i
- **riduzione dei dati e ricostruzione del risultato:** nota la potenza (n - 1)-esima è facile trovare la potenza n-esima:
$$I^n = I * I^{n-1}$$

- **garanzia di terminazione:** n cala di 1 ad ogni chiamata, quindi arriva certamente a 0

Record di attivazione in chiamate ricorsive

Quando chiamo una funzione alloco il record di attivazione della funzione

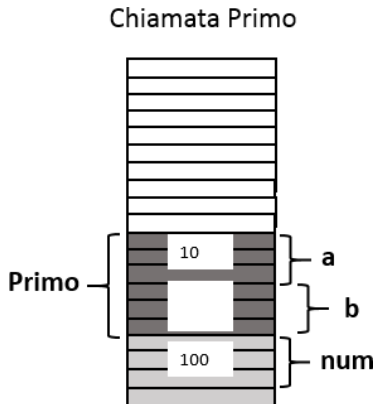
Quindi nel caso della ricorsione ad ogni chiamata allocherò il relativo record di attivazione occupando posto nello stack

Record di attivazione: ricorsione(1)

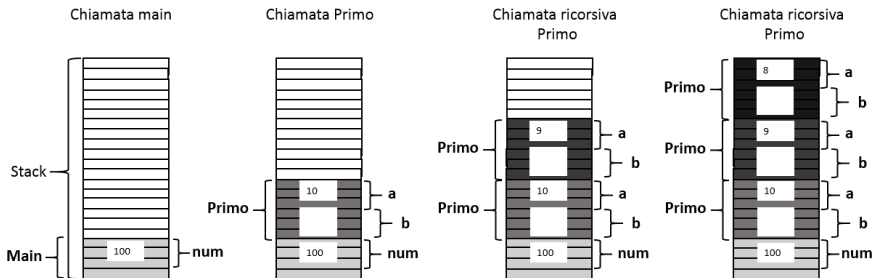
[Esempio in linguaggio C]

```
void Primo(int a){  
    int b;  
  
    if (a > 0)  
        Primo(a-1);  
    return;}  
  
void Secondo(float c){  
}
```

```
int main() {  
    int num=100;  
    Primo(10);  
    Secondo(num);  
}
```



Record di attivazione: ricorsione(2)



[Nota] Cosa succede se un parametro è passato per indirizzo?

Iterazione e ricorsione

Calcolo del fattoriale con tecnica iterativa e ricorsiva

[Fattoriale iterativo]

```
int fattoriale (int n) {  
    int i, f;  
    f = 1;  
    for (i = 1; i <= n; i++) {  
        f = f * i;  
    } return f;  
}
```

[Ricorsiva]

```
int fattoriale (int n) {  
    if (n == 0) return 1;  
    else return (n * fattoriale(n - 1));  
}
```

Ricorsione multipla

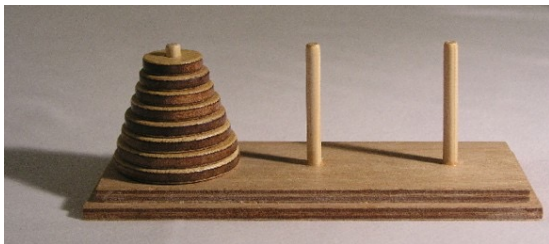
- Si ha ricorsione multipla quando un'attivazione di funzione causa più di una attivazione ricorsiva della stessa funzione
- n-esimo numero di Fibonacci

[Fibonacci ricorsivo]

```
int F(n){  
    if (n==0) return 0;  
    if (n==1) return 1;  
    if (n>1) return F(n-2) + F(n-1);  
}
```

Ricorsione e intrattabilità

- Un es. di problema intrattabile: la Torre di Hanoi



Torre di Hanoi

- Tre paletti e un certo numero di dischi di grandezza decrescente
- Il gioco inizia con tutti i dischi incolonnati su un paletto in ordine decrescente, in modo da formare un cono.
- Lo scopo del gioco e' portare tutti dischi sull'ultimo paletto, potendo spostare solo un disco alla volta e potendo mettere un disco solo su un altro disco piu' grande, mai su uno piu' piccolo.
- il numero minimo di mosse necessarie per completare il gioco è $2^n - 1$, dove n è il numero di dischi.
- Di conseguenza, secondo la leggenda, i monaci di Hanoi dovrebbero effettuare almeno 18.446.744.073.709.551.615 mosse prima che il mondo finisca (una mossa al secondo 585 miliardi di anni), essendo $n = 64$

Torre di Hanoi e ricorsione(1)

- Problema caratterizzato dalla sua intrattabilità
- **Soluzione naturale ricorsiva:** consideriamo i paletti da sinistra verso destra come P_1 , P_2 e P_3 e i dischi da 1 il più piccolo a n il più grande
 - Sposta i primi $n - 1$ dischi da P_1 a P_2 usando P_3 come appoggio
 - Sposta il disco n da P_1 a P_3
 - Sposta $n - 1$ dischi da P_2 a P_3

Torre di Hanoi e ricorsione(2)

- Considero il numero di dischi n il numero del palo sorgente s il numero del palo destinazione d e il palo di appoggio a
- Partendo da un certo numero di dischi e scegliendo sorgente e destinazione possiamo scrivere la seguente funzione ricorsiva

[Hanoi ricorsivo]

```
void hanoi(n, s, d, a) {  
    if (n == 1)  
        muoviDisco(s, d);  
    else {  
        hanoi(n-1, s, a, d);  
        muoviDisco(s, d);  
        hanoi(n-1, a, d, s);  
    }  
}
```

Iterazione o ricorsione

- Come facciamo ad accorgerci se una ricorsione è corretta o **ben definita**?
- Se ad ogni volta che applico la ricorsione sono significativamente più vicino al **caso base**
- Si dice che a definizione non è **circolare** ma converge
- Solitamente le soluzioni ricorsive sono più eleganti, sono più vicine alla definizione del problema
- Non è detto che siano più efficienti
- L'utilizzo di algoritmi ricorsivi costituisce l'opzione più naturale ed intuitiva per operare su strutture dinamiche definite in modo ricorsivo (esempio alberi)

Relazioni di ricorrenza - strutture dati

- Poter derivare delle relazioni di ricorrenza è importante
 - Permette di organizzare il codice
- **Relazione tra tipo di dato strutturato e relazioni di ricorrenza:**
 - array -> cicli
 - Cicli che scorrono indici (es for)
 - liste -> ricorsione
 - In generale tipi di dato dinamici con allocazione sullo heap
- La ricorsione attraverso le chiamate a sottoprogrammi, intrinsecamente genera una sorta di struttura dinamica
 - record di attivazione allocati man mano

Esempio

Scrivere un programma che legge in ingresso una sequenza lunga a piacere di numeri uno alla volta e la ritorni in ordine inverso terminando quando legge 0

Es ingresso

1
E
R
F

Uscita:
F R E 1

Soluzione

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  /* run this program using the console pauser or a
5
6  void leggi(){
7      int n;
8      scanf("%d",&n);
9      if (n)
10         leggi();
11     printf("%d ",n);
12 }
13
14 ► int main(int argc, char *argv[]) {
15     leggi();
16     return 0;
17 }
18
```

Soluzione

Terminale:\>

1

Prima chiamata a leggi

Record
attivo



n=1

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  /* run this program using the console pauser or a
5
6  void leggi(){
7      int n;
8      scanf("%d",&n);
9      if (n)
10         leggi();
11     printf("%d ",n);
12 }
13
14 int main(int argc, char *argv[]) {
15     leggi();
16     return 0;
17 }
18
```

Soluzione

Terminale:\>

1

2

Seconda chiamata a leggi

Record
attivo



n=2

n=1

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  /* run this program using the console pauser or a
5
6  void leggi(){
7      int n;
8      scanf("%d",&n);
9      if (n)
10         leggi();
11     printf("%d ",n);
12 }
13
14 int main(int argc, char *argv[]) {
15     leggi();
16     return 0;
17 }
18
```

Soluzione

Terminale:\>

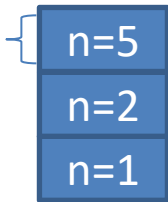
1

2

5

terza chiamata a leggi

Record
attivo



```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  /* run this program using the console pauser or a
5
6  void leggi(){
7      int n;
8      scanf("%d",&n);
9      if (n)
10         leggi();
11     printf("%d ",n);
12 }
13
14 int main(int argc, char *argv[]) {
15     leggi();
16     return 0;
17 }
18
```


Soluzione

Terminale:\>

1

2

5

terza chiamata a leggi

Record
attivo

n=5

n=2

n=1

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  /* run this program using the console pauser or a
5
6  void leggi(){
7      int n;
8      scanf("%d",&n);
9      if (n)
10         leggi();
11     printf("%d ",n);
12 }
13
14 int main(int argc, char *argv[]) {
15     leggi();
16     return 0;
17 }
18
```

Soluzione

Terminale:\>

1

2

5

0

fine fase di winding

quarta chiamata a leggi

Record
attivo



n=0

n=5

n=2

n=1

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  /* run this program using the console pauser or a
5
6  void leggi(){
7      int n;
8      scanf("%d",&n);
9      if (n)
10         leggi();
11     printf("%d ",n);
12 }
13
14 int main(int argc, char *argv[]) {
15     leggi();
16     return 0;
17 }
18
```

Soluzione

Terminale:\>

1

2

5

0

0

Fase di unwinding

Record
attivo



n=5

n=2

n=1

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  /* run this program using the console pauser or a
5
6  void leggi(){
7      int n;
8      scanf("%d",&n);
9      if (n)
10         leggi();
11     printf("%d ",n);
12 }
13
14 int main(int argc, char *argv[]) {
15     leggi();
16     return 0;
17 }
18
```

Soluzione

Terminale:\>

1

2

5

0

0 5

Fase di unwinding

Record
attivo

n=2

n=1

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  /* run this program using the console pauser or a
5
6  void leggi(){
7      int n;
8      scanf("%d",&n);
9      if (n)
10         leggi();
11     printf("%d ",n);
12 }
13
14 int main(int argc, char *argv[]) {
15     leggi();
16     return 0;
17 }
18
```

Soluzione

Terminale:\>

1

2

5

0

0 5 2

Fase di unwinding

Record
attivo

n=1

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  /* run this program using the console pauser or a
5
6  void leggi(){
7      int n;
8      scanf("%d",&n);
9      if (n)
10         leggi();
11     printf("%d ",n);
12 }
13
14 int main(int argc, char *argv[]) {
15     leggi();
16     return 0;
17 }
18
```

Soluzione

Terminale:\>

1

2

5

0

0 5 2 1

Fase di unwinding

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  /* run this program using the console pauser or a
5
6  void leggi(){
7      int n;
8      scanf("%d",&n);
9      if (n)
10         leggi();
11     printf("%d ",n);
12 }
13
14 int main(int argc, char *argv[]) {
15     leggi();
16     return 0;
17 }
18
```

Esempio Hanoi

Scrivere il programma che risolve la torre di Hanoi in c

Lezione 13 – command line

Corso — Programmazione

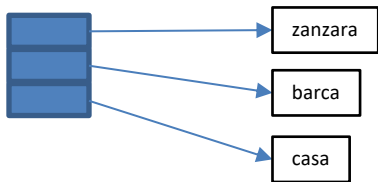
Marco Anisetti

e-mail: marco.anisetti@unimi.it

web: <http://homes.di.unimi.it/anisetti/>

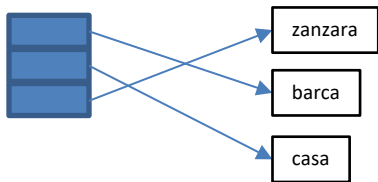
Array di stringhe

- Sono array di puntatori a carattere
- Semplificano la vita in certi contesti applicativi
- Es. immaginiamoci di avere un array di stringhe che vanno ordinate, basta spostare l'ordine dei puntatori non serve riallocare le intere stringhe



Array di stringhe

- Sono array di puntatori a carattere
- Semplificano la vita in certi contesti applicativi
- Es. immaginiamoci di avere un array di stringhe che vanno ordinate, basta spostare l'ordine dei puntatori non serve riallocare le intere stringhe



La funzione main

Anche il main è una funzione... ma chiamata dal sistema operativo

- ha dei dati, descritti dai parametri formali nell'intestazione
- ha un risultato, descritto dal tipo restituito nell'intestazione

A differenza delle altre però

- il numero e tipo dei parametri non è noto a priori
- il risultato è sempre intero

Rimangono quindi delle questioni aperte

- come si definisce il numero e tipo dei parametri?
- come si passano i parametri attuali?
- a che serve e come si recupera il risultato?

I parametri formali ed attuali

*int main (int argc, char *argv[])*

I parametri formali non sono definiti direttamente

- `int argc`, che e il numero dei parametri
- `char *argv[]`, che e un vettore di lunghezza `argc` composto da stringhe che descrivono i parametri

I parametri attuali vengono passati attraverso la linea di comando

- `argv[0]` e il nome del programma
- `argv[1]` e il primo parametro
- `argv[2]` e il secondo parametro

I parametri formali ed attuali

*int main (int argc, char *argv[])*

I parametri formali non sono definiti direttamente

- `int argc`, che e il numero dei parametri
- `char *argv[]`, che e un vettore di lunghezza `argc` composto da stringhe che descrivono i parametri

I parametri attuali vengono passati attraverso la linea di comando

- `argv[0]` e il nome del programma
- `argv[1]` e il primo parametro
- `argv[2]` e il secondo parametro

Esempio

Anche gcc (il compilatore) è un programma

Se da terminale si scrive la linea di comando

```
gcc codice.c -o codice.exe
```

il programma gcc riceve i seguenti parametri

- argc vale 4
- argv[0] vale "gcc"
- argv[1] vale "codice.c"
- argv[2] vale "-o"
- argv[3] vale "codice.exe"

Risultato

Il risultato di un programma C è sempre un intero

La libreria `stdlib.h` definisce le costanti simboliche

- `EXIT SUCCESS (0)` da usare se il programma ha avuto successo
- `EXIT FAILURE (-1)` da usare in caso di errore

Si usa specificare il tipo di errore definendo altre costanti Simboliche.

Il valore numerico viene restituito al sistema operativo

- alcuni ambienti di compilazione lo indicano all'utente
- i le batch o script lo possono usare

Esempio

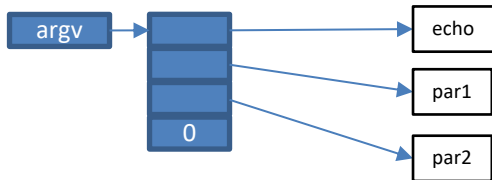
Scrivere un programma che fa l'echo delle stringhe che riceve in ingresso da riga di comando

Esempio

Scrivere un programma che fa l'echo delle stringhe che riceve in ingresso da riga di comando

```
#include <stdio.h>
```

```
int main(int argc, char *argv[])  
{  
    int i;  
    //printf("%s\n",argv[0]);  
    for (i=1; i < argc; i++)  
        printf("%s ", argv[i]);  
    printf("\n");  
    return 0;  
}
```



Se non passassi nulla?

Esempio

Scrivere un programma che riceve due numeri da linea di comando e ne calcola la somma

Esempio

Scrivere un programma che riceve due numeri da linea di comando e ne calcola la somma

```
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char *argv[])
{
    if (argc < 3)
    {
        printf("Errore: mancano gli argomenti.\n");
        return -1;
    }

    printf("%s+%s=%d\n",argv[1],argv[2],atoi(argv[1])+atoi(argv[2]));
    return 0;
}
```